Supplementary Material for "Conflict-Complete Retrieval: Returning Checkable Contradictions from Bounded Agent Memory"

Anonymous submission

Scope

This supplement expands proofs and records the reviewer-facing reproduction contract. It does not introduce an additional headline claim or repair an argument missing from the main paper. The main paper remains self-contained. All confirmatory results concern deterministic synthetic fixtures under an oracle schema. The public-data analysis remains an exploratory topology audit, not a prevalence estimate or a source of semantic conflict labels.

Assumption Ledger

The guarantees use the following assumptions.

1. The premise store is finite after applying the confidence threshold. Assertions have exact RDF-term identity and retain their source identifiers.
2. Each keyed class has one supplied key predicate. Key equality is exact; no entity linker, string normalization, or probabilistic match is inferred.
3. Only schema-declared single-valued predicates can generate the conflicts studied here. Time intervals must overlap and scopes must agree unless an annotation is explicitly unspecified.
4. Query relevance is determined by supplied entity seeds and requested predicates. Conflict completeness requires one complete minimal witness for every relevant conflict, not every alternative derivation.
5. The maintained index is evaluated on the current insertion-oriented snapshot. Deletion, persistence, concurrent mutation, and crash recovery are outside the implementation contract.

Expanded Correctness Arguments

Detector Soundness and Completeness

Let direct co-keying connect two typed records that share the exact key value, and let its reflexive, symmetric, transitive closure be $\sim_K$. Propagation copies only schema-constrained values within one $\sim_K$ class. Therefore a derived unequal-value pair has endpoints in one semantic entity class and satisfies the predicate, confidence, time, and scope guards. This proves soundness.

For completeness, take any semantic conflict. Its endpoint records are in one finite $\sim_K$ class, so a finite path of direct co-key steps connects them. Repeated propagation along the path carries both constrained values to a common representative, where the unequal-value rule fires. The finite source leaf set of that derivation contains a minimal subset: repeatedly delete a leaf whenever entailment remains. Termination is immediate because the leaf set is finite. The remaining subset is a witness by definition.

Fixed-Hop IRI Incompleteness

Fix any finite hop limit k. Construct typed records x and y whose key objects are the same literal, whose constrained values disagree, and for which no triple has an IRI-object path between x and y. A query seeded at x has a relevant direct-key conflict because exact literal-key equality places the records in one $\sim_K$ class. Its logical witness contains the two type, two key, and two value assertions.

The IRI-only graph omits literals as vertices. Consequently x and y are in different connected components, not merely more than k hops apart. Any policy restricted to the k-hop component of x omits the type, key, and value premises incident to y and cannot contain the witness. Since the construction works for every finite k, no fixed finite radius gives the claimed guarantee for this policy class. Literal-aware and key-aware policies are outside the proposition.

CWI Contract and Witness Minimality

After all assertions in a snapshot have been processed, a record belongs to a key bucket exactly when its type and key premises are present. Union–find therefore represents precisely the connected components of the record–bucket graph, which are the $\sim_K$ classes. Within each class, the index compares only schema-constrained values and applies the same time/scope guard as the formal semantics. Hence its materialized conflict set is sound and complete for the finite snapshot.

For a reported cross-record conflict, breadth-first search selects a shortest record–bucket path between the two endpoint records. The certificate contains both endpoint values and, for every path edge, the type and key premises that justify the edge. Removing either endpoint value removes the

Family	Instances	Conflicts	Mechanism
D20	14	8	same/direct key
XR-small	16	12	direct key + distractors
XR-scale	64	60	scaled direct key
Chain-2	41	10	two co-key links
Chain-3	25	6	three co-key links
H3-NK1	16	12	hierarchy/non-key
H3-NK3	16	12	more non-key predicates
TauSig	60	12	time/scope guards
Total	252	132	

Table 1: The deterministic confirmatory fixture matrix.

unequal-value premise. Removing any path premise breaks that co-key edge; because the path is shortest and the certificate contains no alternative path, the endpoints are no longer connected inside the certificate. Thus no proper subset entails the conflict. A chain of m direct co-key links has $m+1$ type assertions, $2m$ key assertions, and two endpoint values, for $3m+3$ logical assertions. A same-record conflict needs only the two unequal values.

Insertion order does not change the final snapshot relation. A record enters a bucket only once its required type and key are both known; a later premise triggers the same membership update that an earlier premise would have triggered. Later constrained values and annotations re-evaluate their touched class. The tests exercise permutations in which types, keys, values, and annotations arrive before and after one another.

Evaluation Matrix

For every instance, the shipped manifest records the query, expected label, fixture identifier, and stable source data. Exact and maintained systems are scored by a checker over the assertions they return. A positive counts only when the returned context contains a complete source-grounded witness. Negative controls include different entities, shared non-key values, disjoint time intervals, and unequal scopes.

The retrieval table in the main paper selects the smallest tested configuration that reaches complete recall within a family. The tested record-level depths are 1–4; relevance methods use $K \in \{1, 2, 4, \ldots, 512\}$. Reciprocal-rank fusion uses 60. Means and maxima are over conflict queries. The scale companion uses one unrecorded warm-up and 11 measured repetitions and reports medians and interquartile ranges. These are descriptive system measurements, not statistical-significance claims.

Artifact Contract

The anonymous code/data archive contains:

- TypeScript source and focused tests for the maintained index, exact comparators, adaptive and on-demand witnesses, retrieval policies, instance scoring, and scale ledger;
- all eight synthetic fixture families and their instance manifests;
- frozen paper-facing outputs for exact, maintained, on-demand, sparse, dense, curve, and scale analyses;
- the pinned Python environment and runner for the dense baseline; and
- the external topology-audit script and its frozen report.

The archive deliberately omits repository history, internal planning documents, hosted-reader runs excluded from the paper, service credentials, caches, installed dependencies, compiled output, and submission-production files. Dense re-execution requires obtaining the cited third-party encoder weights; frozen dense outputs are included so the reported table remains auditable without that download. Timing will vary by machine and should be regenerated only as a systems check, not expected to match byte for byte. For double-blind review, the project-owned RDF namespace is replaced consistently in source, fixtures, and frozen textual outputs by a reserved `example.org` namespace. This name-only substitution changes no graph structure, label, measurement, or scientific value.

Additional Limitations

Shortest-witness selection is per conflict and need not minimize the union of certificates for a multi-conflict query. Materialized unequal-value pairs can be quadratic within a large equivalence class. The scale generator keeps class size bounded and therefore does not expose that worst case. The exploratory public-data audit closes supplied gold match edges transitively, but unequal publication-year strings may represent versions, linkage errors, or metadata errors rather than semantic contradictions. None of these strings enters the confirmatory label set.